Q-Q plots: the reference line, the exponential, and a short right tail

Notes for the questions from class

Narek Sahakyan

Three questions came up while we were on the Q-Q slides.

1. How does ggplot produce the red line, and when is that line $y = x$?
2. What is the exponential distribution in the `rexp` / `qexp` example, and how is it different from the Poisson?
3. If some points on the right of a Q-Q plot lie under the red line, what does that say about the distribution?

What is on each axis

Sort the sample: $x_{(1)} \le x_{(2)} \le \cdots \le x_{(n)}$.

Give the i-th point a probability. For $n > 10$, ggplot uses

$$p_i = \frac{i - 0.5}{n}.$$

(The R function is `ppoints`. For $n \le 10$ it uses a slightly different fraction, $(i - 0.375)/(n + 0.25)$.)

The point is

$$\left(q_{\text{theory}}(p_i),\ x_{(i)} \right).$$

The horizontal coordinate is the quantile the reference distribution predicts. The vertical coordinate is the quantile the sample actually has. By default the reference is the standard normal, so the horizontal axis is a z-score: `qnorm(p)`.

Every Q-Q plot below is calculated the same way. The table keeps the first point, the last point, and the points nearest $p = 0.25$, 0.50, and 0.75. The line is not fitted to those rows. It uses the exact probabilities 0.25 and 0.75.

```
qq_values <- function(sample, distribution = stats::qnorm, dparams = list()) {
  y <- sort(sample[is.finite(sample)])
  n <- length(y)
  p <- stats::ppoints(n)
  theoretical <- do.call(distribution, c(list(p = p), dparams))
  pick <- unique(c(1L,
                   which.min(abs(p - 0.25)),
                   which.min(abs(p - 0.50)),
                   which.min(abs(p - 0.75)),
                   n))
  points <- data.frame(
```

```
    i = pick,
    p = signif(p[pick], 4),
    theoretical = signif(theoretical[pick], 4),
    sample = signif(y[pick], 4)
  )
  xq <- as.numeric(do.call(distribution, c(list(p = c(0.25, 0.75)), dparams)))
  yq <- as.numeric(stats::quantile(y, c(0.25, 0.75)))
  slope <- (yq[2] - yq[1]) / (xq[2] - xq[1])
  intercept <- yq[1] - slope * xq[1]
  line <- data.frame(
    x25 = xq[1], y25 = yq[1], x75 = xq[2], y75 = yq[2],
    slope = slope, intercept = intercept
  )
  list(points = points, line = signif(line, 4))
}
```

How the red line is built

`geom_qq_line` does not fit a regression through every point, and it does not force the line $y = x$.

It takes two probabilities, 0.25 and 0.75.

- Horizontal coordinates: the reference distribution's own quartiles.
- Vertical coordinates: the sample's quartiles, `quantile(sample, c(0.25, 0.75))`.

$$\text{slope} = \frac{y_{0.75} - y_{0.25}}{x_{0.75} - x_{0.25}}, \qquad \text{intercept} = y_{0.25} - \text{slope} \times x_{0.25}.$$

The drawn line is $y = \text{intercept} + \text{slope} \times x$, extended across the range of the theoretical quantiles. That is the same slope and intercept the lecture computes from `ggplot_build()$data[[2]]`.

You get the 45° line $y = x$ only when those two quartiles already match: slope $= 1$ and intercept $= 0$. That happens when the reference distribution is on the same scale as the data.

Standard normal: the line really is $y = x$

The standard normal quartiles are

```
z25 <- qnorm(0.25)
z75 <- qnorm(0.75)
c(z25 = z25, z75 = z75, gap = z75 - z25)
```

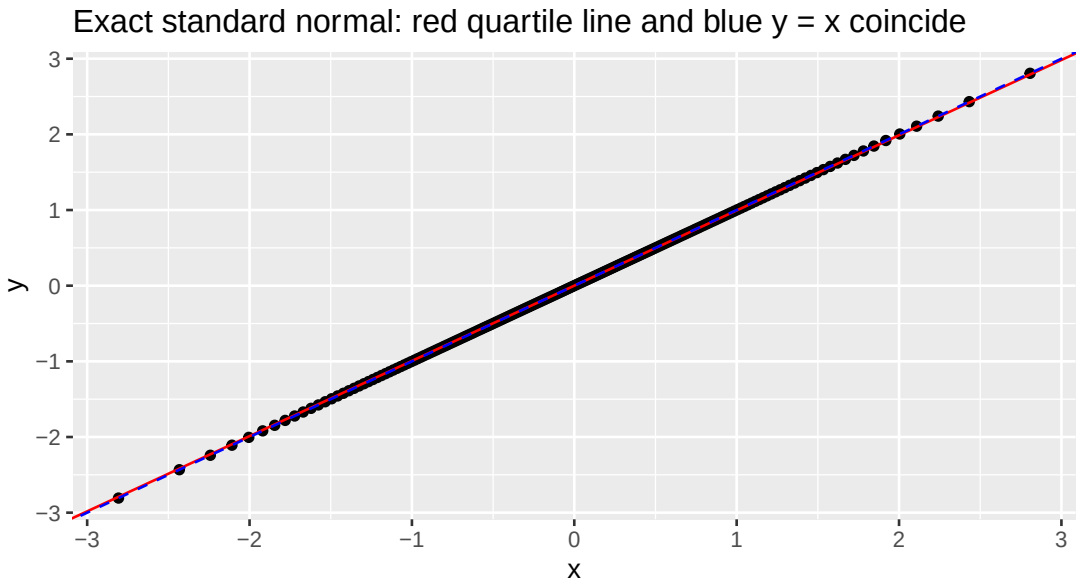
```
##        z25        z75        gap
## -0.6744898  0.6744898  1.3489795
```

If the sample is exactly standard normal, its quartiles are these same two numbers. Then

$$\text{slope} = \frac{0.674 - (-0.674)}{0.674 - (-0.674)} = 1, \qquad \text{intercept} = -0.674 - 1 \times (-0.674) = 0.$$

The line through $(-0.674, -0.674)$ and $(0.674, 0.674)$ is $y = x$.

```
z <- qnorm(ppoints(200))
ggplot(mapping = aes(sample = z)) +
  geom_qq() +
  geom_qq_line(color = "red") +
  geom_abline(slope = 1, intercept = 0, linetype = 2, color = "blue") +
  ggtitle("Exact standard normal: red quartile line and blue y = x coincide")
```



The sample was built as `qnorm(ppoints(200))`, and the reference is `qnorm`, so the two columns are the same numbers. $p_i = (i - 0.5)/200$. The quartile slope is about 1 and the intercept is 0.

```
calc <- qq_values(z)
knitr::kable(calc$points, linesep = "")
```

i	p	theoretical	sample
1	0.0025	-2.807000	-2.807000
50	0.2475	-0.682400	-0.682400
101	0.5025	0.006267	0.006267
150	0.7475	0.666600	0.666600
200	0.9975	2.807000	2.807000

```
knitr::kable(calc$line, linesep = "")
```

x25	y25	x75	y75	slope	intercept
-0.6745	-0.6706	0.6745	0.6706	0.9942	0

Normal with mean 10 and sd 2: the line is $y = 10 + 2x$

Leave the horizontal axis as a standard normal z-score. A Normal$(10, 2)$ quartile is $10 + 2z$.

```r
y25 <- 10 + 2 * z25
y75 <- 10 + 2 * z75
slope <- (y75 - y25) / (z75 - z25)
intercept <- y25 - slope * z25
c(y25 = y25, y75 = y75, slope = slope, intercept = intercept)
```

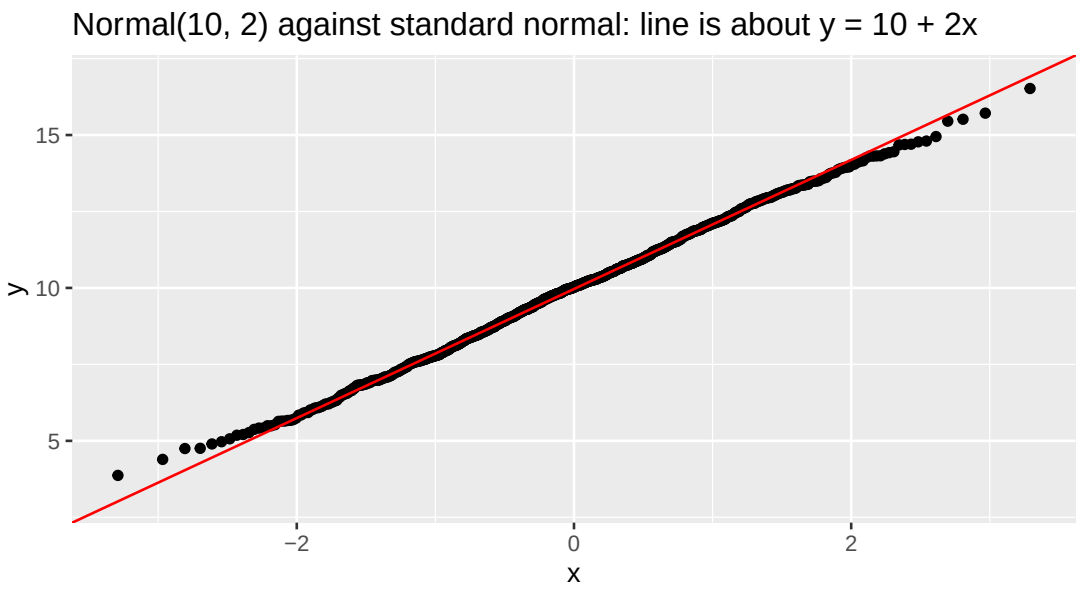
```
##       y25       y75     slope intercept
##   8.65102  11.34898   2.00000  10.00000
```

Rounded to three decimals, $z_{0.25} = -0.674$ and $z_{0.75} = 0.674$, so the gap on the horizontal axis is 1.349. The sample quartiles are 8.651 and 11.349.

$$\text{slope} = \frac{11.349 - 8.651}{1.349} = \frac{2.698}{1.349} = 2, \qquad \text{intercept} = 8.651 - 2 \times (-0.674) = 10.$$

The line is $y = 10 + 2x$. The slope is the standard deviation. The intercept is the mean. This is the sentence on the lecture slide, and it is special to the normal reference: it is not a rule for every distribution.

```r
x <- rnorm(1000, mean = 10, sd = 2)
p_default <- ggplot(mapping = aes(sample = x)) +
  geom_qq() +
  geom_qq_line(color = "red") +
  ggtitle("Normal(10, 2) against standard normal: line is about y = 10 + 2x")
p_default
```




```r
ggplot_build(p_default)$data[[2]][, c("slope", "intercept")]
```

```
##      slope intercept
## 1 2.110503  9.964597
## 2 2.110503  9.964597
```

The simulated slope is near 2 and the intercept is near 10. It is not exactly 2 and 10, because a sample of 1000 is not the population. The horizontal column is still `qnorm(p)`. The vertical column is the sorted `rnorm` draw, about $10 + 2$ times the horizontal value.

```
calc <- qq_values(x)
knitr::kable(calc$points, linesep = "")
```

i	p	theoretical	sample
1	0.0005	-3.291000	3.870
250	0.2495	-0.676100	8.531
501	0.5005	0.001253	10.030
750	0.7495	0.672900	11.390
1000	0.9995	3.291000	16.520

```
knitr::kable(calc$line, linesep = "")
```

x25	y25	x75	y75	slope	intercept
-0.6745	8.541	0.6745	11.39	2.111	9.965

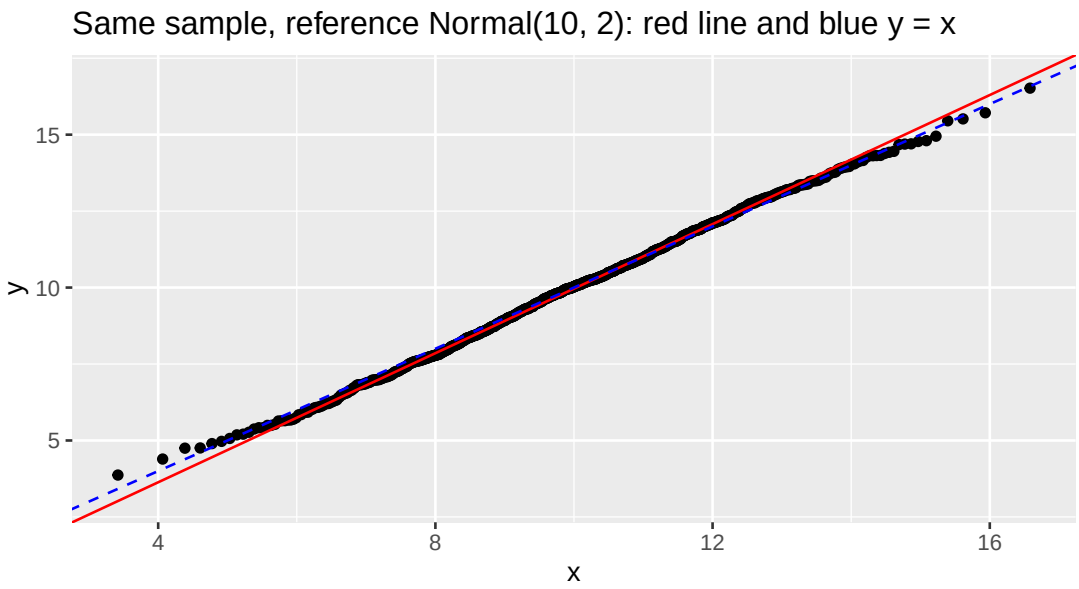
Put the mean and sd into the reference, and the line becomes $y = x$

`dparams = list(mean = 10, sd = 2)` asks for `qnorm(p, mean = 10, sd = 2)` instead of `qnorm(p)`. The horizontal quartiles are now 8.651 and 11.349, the same numbers as the sample. Slope $= 1$, intercept $= 0$.

```
x25 <- qnorm(0.25, mean = 10, sd = 2)
x75 <- qnorm(0.75, mean = 10, sd = 2)
slope2 <- (y75 - y25) / (x75 - x25)
intercept2 <- y25 - slope2 * x25
c(x25 = x25, x75 = x75, slope = slope2, intercept = intercept2)
```

```
##       x25       x75     slope intercept
##   8.65102  11.34898   1.00000   0.00000
```

```
p_scaled <- ggplot(mapping = aes(sample = x)) +
  geom_qq(dparams = list(mean = 10, sd = 2)) +
  geom_qq_line(color = "red", dparams = list(mean = 10, sd = 2)) +
  geom_abline(slope = 1, intercept = 0, linetype = 2, color = "blue") +
  ggtitle("Same sample, reference Normal(10, 2): red line and blue y = x")
p_scaled
```



The sample column is the same draw as the previous plot. `theoretical` is now `qnorm(p, mean = 10, sd = 2)`, so it is on the scale of the data. The reference quartiles match the sample quartiles, and the slope is about 1.

```
calc <- qq_values(x, dparams = list(mean = 10, sd = 2))
knitr::kable(calc$points, linesep = "")
```

i	p	theoretical	sample
1	0.0005	3.419	3.870
250	0.2495	8.648	8.531
501	0.5005	10.000	10.030
750	0.7495	11.350	11.390
1000	0.9995	16.580	16.520

```
knitr::kable(calc$line, linesep = "")
```

x25	y25	x75	y75	slope	intercept
8.651	8.541	11.35	11.39	1.055	-0.5879

So ggplot is not "generating $y = x$ for different distributions." It always connects the two quartiles. $y = x$ is what that connection looks like after the reference distribution has been given the same parameters as the data.

Exponential is a waiting time. Poisson is a count.

These are different variables that describe the same story from two sides.

Suppose emails arrive at a constant average rate λ, independently of one another.

- The **number of emails in one fixed hour** is a Poisson count: $0, 1, 2, ...$
- The **time you wait for the next email** is an exponential number of hours: any positive real value, such as 0.2 or 1.7.

The Poisson probabilities, for mean λ, are

$$P(K = k) = e^{-\lambda}\frac{\lambda^k}{k!}, \qquad k = 0, 1, 2, ...$$

Mean and variance are both λ.

The exponential density, for rate λ, is

$$f(t) = \lambda e^{-\lambda t}, \qquad t \geq 0.$$

The cumulative probability and the quantile used by `qexp` are

$$F(t) = 1 - e^{-\lambda t}, \qquad q(p) = -\frac{\ln(1-p)}{\lambda}.$$

The mean wait is $1/\lambda$. The variance is $1/\lambda^2$. The median is $\ln(2)/\lambda$, which is smaller than the mean, so the distribution is right-skewed. The highest point of the density is at $t = 0$: short waits are the most common, and a very long wait is rare but possible.

There is no factorial and no integer restriction in the exponential formulas. There is no waiting-time t in the Poisson formula.

A numerical comparison with $\lambda = 2$

Poisson, events per hour:

```
k <- 0:4
pois <- exp(-2) * 2^k / factorial(k)
data.frame(k = k, probability = round(pois, 4))
```

```
##   k probability
## 1 0      0.1353
## 2 1      0.2707
## 3 2      0.2707
## 4 3      0.1804
## 5 4      0.0902
```

Check the first two by hand. $e^{-2} \approx 0.1353$, so $P(K = 0) = 0.1353$. $P(K = 1) = 2 \times 0.1353 = 0.2707$. The mean of this distribution is 2, not $1/2$.

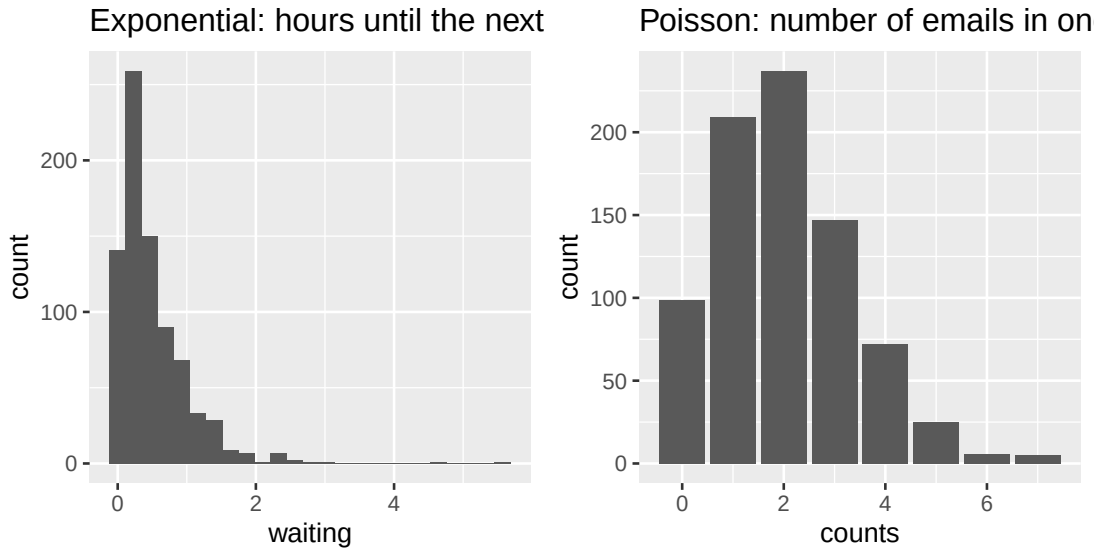
Exponential, hours until the next event, rate 2:

```
p <- c(0.25, 0.50, 0.75)
data.frame(p = p, wait = qexp(p, rate = 2), by_hand = -log(1 - p) / 2)
```

```
##      p      wait   by_hand
## 1 0.25 0.1438410 0.1438410
## 2 0.50 0.3465736 0.3465736
## 3 0.75 0.6931472 0.6931472
```

The median wait is $-\ln(0.5)/2 = 0.693/2 = 0.347$ hours, about 21 minutes. The mean wait is $1/2 = 0.5$ hours. Half of waits are shorter than 21 minutes, but the long ones pull the mean up to 30 minutes.

```
waiting <- rexp(800, rate = 2)
counts <- rpois(800, lambda = 2)
g_wait <- ggplot(data.frame(waiting), aes(waiting)) +
  geom_histogram(bins = 25) +
  ggtitle("Exponential: hours until the next email")
g_count <- ggplot(data.frame(counts), aes(counts)) +
  geom_bar() +
  ggtitle("Poisson: number of emails in one hour")
gridExtra::grid.arrange(g_wait, g_count, ncol = 2)
```



The lecture slide `x_exp <- rexp(10000)` followed by `distribution = stats::qexp` is the left-hand picture. `rexp` draws waiting times. The default rate is 1, so the mean wait is 1. It is not a Poisson sample, and `qexp` is not a Poisson quantile function. The Poisson quantile function is `qpois`.

Hand calculation: when the exponential Q-Q line is $y = x$

Use rate $\lambda = 1$, which is the default of both `rexp` and `qexp`.

$$
\begin{aligned}
q(0.25) &= -\ln(0.75) = 0.288, \\
q(0.50) &= -\ln(0.50) = 0.693, \\
q(0.75) &= -\ln(0.25) = 1.386.
\end{aligned}
$$

```
c(q25 = -log(0.75), q50 = -log(0.50), q75 = -log(0.25))
```

```
##       q25       q50       q75
## 0.2876821 0.6931472 1.3862944
```

If the sample is exactly this exponential, its quartiles equal those three numbers. The slope between the first and third is

$$\frac{1.386 - 0.288}{1.386 - 0.288} = 1.$$

The line is $y = x$. That is why the `rexp(10000)` slide looks like a 45° line: the sample and the reference were both exponential with rate 1.

Now change the rate to $\lambda = 0.5$. The mean wait doubles, from 1 to 2, and every quantile doubles, because of the λ in the denominator:

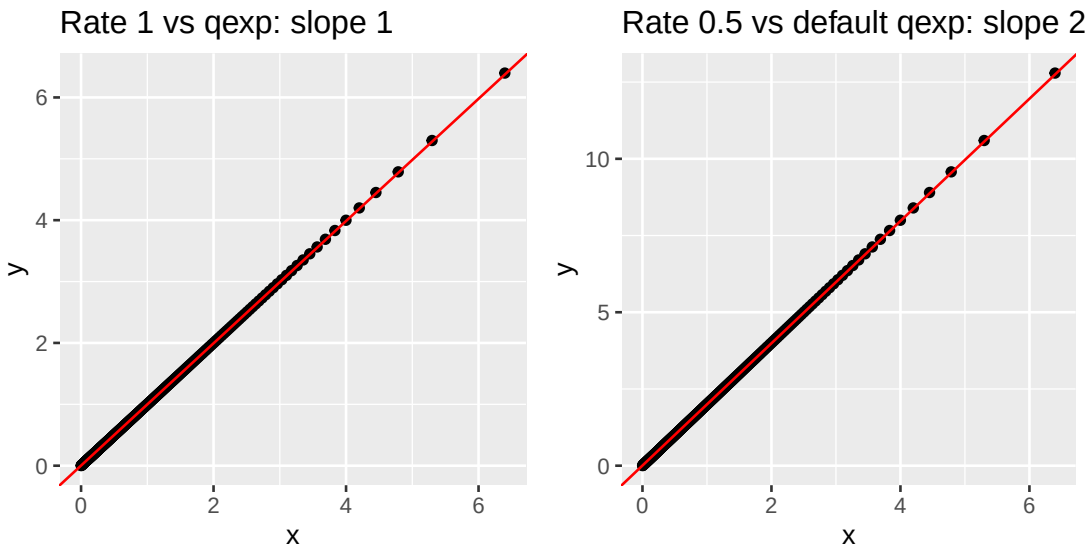
$$q_{0.5}(p) = -\frac{\ln(1-p)}{0.5} = 2 \times (-\ln(1-p)).$$

Compare this sample with the default `qexp`, which still uses rate 1:

$$\text{slope} = \frac{2 \times 1.386 - 2 \times 0.288}{1.386 - 0.288} = 2.$$

The line is $y = 2x$, and 2 is the mean of this exponential. Passing `dparams = list(rate = 0.5)` puts the factor 2 into the horizontal axis as well, and the line returns to $y = x$.

```
exact_rate1 <- qexp(ppoints(300), rate = 1)
exact_rate05 <- qexp(ppoints(300), rate = 0.5)
g1 <- ggplot(mapping = aes(sample = exact_rate1)) +
  geom_qq(distribution = qexp) +
  geom_qq_line(color = "red", distribution = qexp) +
  ggtitle("Rate 1 vs qexp: slope 1")
g2 <- ggplot(mapping = aes(sample = exact_rate05)) +
  geom_qq(distribution = qexp) +
  geom_qq_line(color = "red", distribution = qexp) +
  ggtitle("Rate 0.5 vs default qexp: slope 2")
gridExtra::grid.arrange(g1, g2, ncol = 2)
```



Rate 1: the sample is `qexp(ppoints(300), rate = 1)` and the reference is `qexp` at rate 1, so $q(p) = -\ln(1-p)$. The two columns match and the slope is about 1.

```
calc <- qq_values(exact_rate1, distribution = qexp)
knitr::kable(calc$points, linesep = "")
```

i	p	theoretical	sample
1	0.001667	0.001668	0.001668
76	0.251700	0.289900	0.289900
150	0.498300	0.689800	0.689800
225	0.748300	1.380000	1.380000
300	0.998300	6.397000	6.397000

```
knitr::kable(calc$line, linesep = "")
```

x25	y25	x75	y75	slope	intercept
0.2877	0.2888	1.386	1.383	0.996	0.002272

Rate 0.5: every sample quantile is twice the rate-1 quantile, because $q(p) = -\ln(1-p)/0.5$. The reference on the plot is still rate 1, so the slope is about 2.

```
calc <- qq_values(exact_rate05, distribution = qexp)
knitr::kable(calc$points, linesep = "")
```

i	p	theoretical	sample
1	0.001667	0.001668	0.003336
76	0.251700	0.289900	0.579800
150	0.498300	0.689800	1.380000
225	0.748300	1.380000	2.759000
300	0.998300	6.397000	12.790000

```
knitr::kable(calc$line, linesep = "")
```

x25	y25	x75	y75	slope	intercept
0.2877	0.5776	1.386	2.766	1.992	0.004544

The budget slide is the same idea, in dollars

`budget_adjusted` is a positive continuous amount, so an exponential reference is a shape to try. It is not a count, so a Poisson reference would be the wrong family.

```
budget <- read.csv("Data/movies.csv")$budget_adjusted
p_budget <- ggplot(data.frame(budget), aes(sample = budget)) +
  geom_qq(distribution = qexp) +
  geom_qq_line(color = "red", distribution = qexp) +
  ggtitle("Adjusted budget against a rate-1 exponential")
line_budget <- ggplot_build(p_budget)$data[[2]]
line_budget[, c("slope", "intercept")]
```

```
##      slope intercept
## 1 52098492    313937
## 2 52098492    313937
```

```
c(mean_budget = mean(budget), max_theoretical = max(qexp(ppoints(length(budget)))))
```

```
##     mean_budget max_theoretical
## 51910464.181037        8.669399
```

The horizontal column is $-\ln(1-p)$ for a rate-1 exponential. The last of those, for 2911 films, is about 8.7. The sample column is the budget in dollars. The quartile slope is the scale of those budgets.

```
calc <- qq_values(budget, distribution = qexp)
knitr::kable(calc$points, linesep = "")
```

i	p	theoretical	sample
1	0.0001718	0.0001718	290
728	0.2499000	0.2876000	15300000
1456	0.5000000	0.6931000	35380000
2184	0.7501000	1.3870000	72600000
2911	0.9998000	8.6690000	354700000

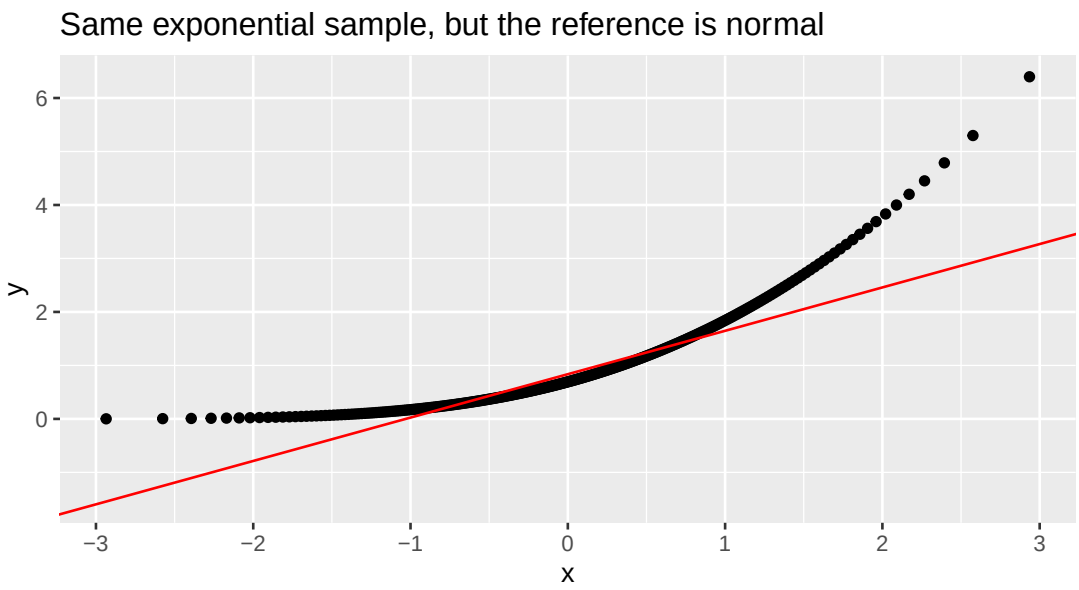
```
knitr::kable(calc$line, linesep = "")
```

x25	y25	x75	y75	slope	intercept
0.2877	15300000	1.386	72540000	52100000	313900

Read the two axes separately. The horizontal coordinates are $-\ln(1-p)$ for a rate-1 exponential. The largest of those, for 2911 films, is only about 8.7. That number is not a budget. The vertical coordinates are budgets in dollars. The quartile slope is about 5.2×10^7, close to the mean budget, for the same reason the rate-0.5 exponential had slope 2: against a rate-1 reference, the slope estimates the scale of the data. The line is not $y = x$, and it should not be. To ask whether the line would be $y = x$, the reference rate would have to be 1/mean, about $1/(5.2 \times 10^7)$.

One more contrast with the default Q-Q plot: an exponential sample plotted against a **normal** reference does not lie on the line. The right side bends up, because an exponential is right-skewed relative to a normal. The lecture avoids that by changing `distribution` to `qexp`.

```
ggplot(mapping = aes(sample = exact_rate1)) +
  geom_qq() +
  geom_qq_line(color = "red") +
  ggtitle("Same exponential sample, but the reference is normal")
```



This is the same rate-1 exponential vector, but the reference is back to `qnorm`. The last sample value is a long wait, much larger than the normal quantile at that probability, so the right-hand point sits above the quartile line.

```
calc <- qq_values(exact_rate1)
knitr::kable(calc$points, linesep = "")
```

i	p	theoretical	sample
1	0.001667	-2.935000	0.001668
76	0.251700	-0.669300	0.289900
150	0.498300	-0.004178	0.689800
225	0.748300	0.669300	1.380000
300	0.998300	2.935000	6.397000

```
knitr::kable(calc$line, linesep = "")
```

x25	y25	x75	y75	slope	intercept
-0.6745	0.2888	0.6745	1.383	0.8111	0.8359

A point on the right that lies under the line

The right side of the horizontal axis is the high theoretical quantiles: the large values the reference distribution expects.

A point there has coordinates $(q_{\text{theory}}(p),\ x_{(p)})$ with p near 1. It lies under the red line when the sample quantile is smaller than the line's height at that theoretical quantile.

Against a standard normal the line is $y = x$, so "under the line" means

$$x_{(p)} < q_{\text{normal}}(p).$$

Take $p = 0.95$. The standard normal quantile is

```
qnorm(0.95)
```

```
## [1] 1.644854
```

which is 1.645. The line's height there is also 1.645.

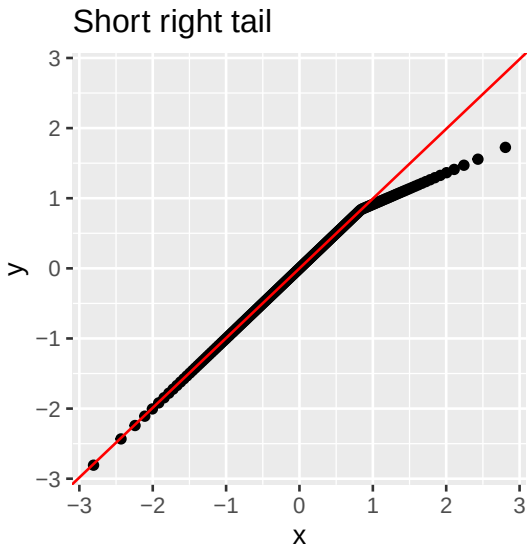
Suppose the sample's own 0.95 quantile is 1.10. The plotted point is $(1.645,\ 1.10)$. It sits $1.645 - 1.10 = 0.545$ under the line. In words: 95% of this sample is at or below 1.10, while a standard normal with this center and spread would still be climbing until 1.645. The largest observations are not large enough. The right tail is lighter, or shorter, than the reference.

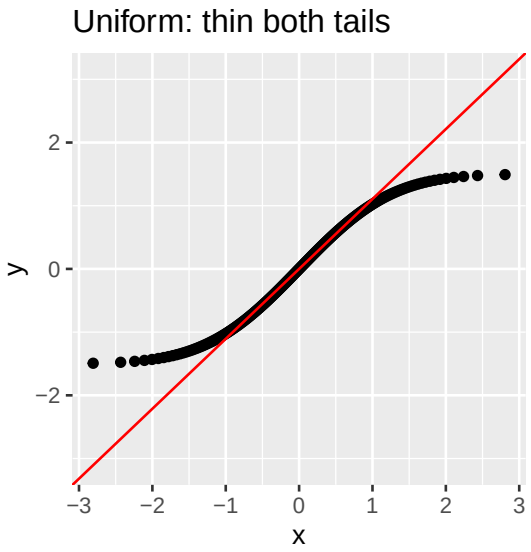
That is one feature, not a distribution name. Read the left side before naming a shape.

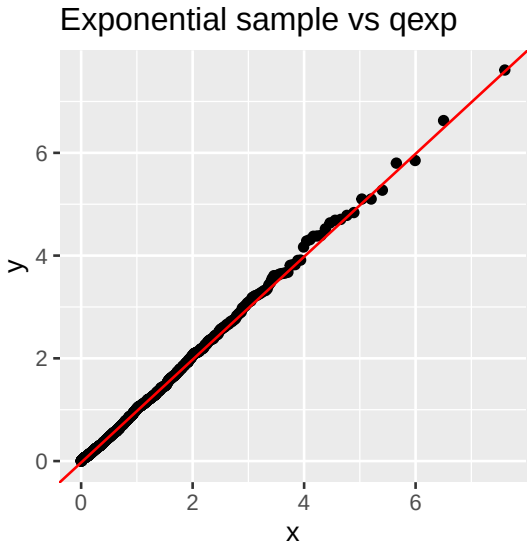
- Right side **under** the line, left side **over** the line: both tails are thin. The sample runs out of extreme values on both ends. A uniform sample against a normal reference looks like this.
- Right side **under** the line, and the left side also **under** the line: the whole sample sits lower than the reference expects. Against a normal, that is the left-skew pattern from the lecture: sample quantiles are lower than the theoretical ones.
- Right side **over** the line: the largest observations are larger than the reference expects. Against a normal, that is a heavier right tail, and it is the usual look of a right-skewed sample.

```
n <- 200
p <- ppoints(n)
normal_sample <- qnorm(p)
short_right <- normal_sample
short_right[p > 0.8] <- 0.84 + 0.45 * (normal_sample[p > 0.8] - 0.84)
uniform_sample <- qunif(p, min = -1.5, max = 1.5)
set.seed(27)
exponential_sample <- rexp(1000, rate = 1)

g_short <- ggplot(mapping = aes(sample = short_right)) +
  geom_qq() +
  geom_qq_line(color = "red") +
  ggtitle("Short right tail")
g_unif <- ggplot(mapping = aes(sample = uniform_sample)) +
  geom_qq() +
  geom_qq_line(color = "red") +
  ggtitle("Uniform: thin both tails")
g_exp <- ggplot(mapping = aes(sample = exponential_sample)) +
  geom_qq(distribution = qexp) +
  geom_qq_line(color = "red", distribution = qexp) +
  ggtitle("Exponential sample vs qexp")
gridExtra::grid.arrange(g_short, g_unif, g_exp, ncol = 2)
```







Short right tail, against `qnorm`. At the right end the sample value is smaller than `qnorm(p)`, so that point is under the line. The left end is still close to the normal quantile.

```
calc <- qq_values(short_right)
knitr::kable(calc$points, linesep = "")
```

i	p	theoretical	sample
1	0.0025	-2.807000	-2.807000
50	0.2475	-0.682400	-0.682400
101	0.5025	0.006267	0.006267
150	0.7475	0.666600	0.666600
200	0.9975	2.807000	1.725000

```
knitr::kable(calc$line, linesep = "")
```

x25	y25	x75	y75	slope	intercept
-0.6745	-0.6706	0.6745	0.6706	0.9942	0

Uniform, against `qnorm`. The smallest uniform value is not as small as `qnorm(p)`, so the left point is above the line. The largest is not as large, so the right point is under the line.

```
calc <- qq_values(uniform_sample)
knitr::kable(calc$points, linesep = "")
```

i	p	theoretical	sample
1	0.0025	-2.807000	-1.4920
50	0.2475	-0.682400	-0.7575
101	0.5025	0.006267	0.0075
150	0.7475	0.666600	0.7425
200	0.9975	2.807000	1.4930

```
knitr::kable(calc$line, linesep = "")
```

x25	y25	x75	y75	slope	intercept
-0.6745	-0.7463	0.6745	0.7463	1.106	0

Exponential sample against `qexp` at rate 1. The horizontal value is $-\ln(1-p)$. The sample column stays near that value, including at the right end.

```
calc <- qq_values(exponential_sample, distribution = qexp)
knitr::kable(calc$points, linesep = "")
```

i	p	theoretical	sample
1	0.0005	0.0005001	0.0008931
250	0.2495	0.2870000	0.2589000
501	0.5005	0.6941000	0.6515000
750	0.7495	1.3840000	1.3570000
1000	0.9995	7.6010000	7.6120000

```
knitr::kable(calc$line, linesep = "")
```

x25	y25	x75	y75	slope	intercept
0.2877	0.2589	1.386	1.359	1.002	-0.02928

The left-hand plot was built from exact normal quantiles, then the upper ones were pulled down. One of those points, the plotting position nearest 0.95:

```r
i <- which.min(abs(p - 0.95))
theoretical <- qnorm(p[i])
observed <- short_right[i]
data.frame(
  p = p[i],
  theoretical = theoretical,
  sample = observed,
  line = theoretical,
  below_by = theoretical - observed
)
```

```
##        p theoretical   sample     line  below_by
## 1 0.9475    1.621082 1.191487 1.621082 0.4295952
```

The sample value is smaller than the line. Several neighboring points do the same, so the right-hand end of the cloud sags.

The right-hand plot is the other common version of "under the line on the right." A uniform has no long tail at either end. On the right the points fall below the normal line. On the left they fall above it, because the smallest uniform values are not as small as a normal would produce.

Do not call this pattern exponential. An exponential, judged against a normal line, bends the other way: the right side goes **above** the line. The third plot is the comparison that belongs with an exponential claim. `exponential_sample` is `rexp(1000, rate = 1)`, and the reference is `qexp` at the same rate, so the theoretical quantiles are $-\ln(1-p)$. The points follow the red line. A short right tail does not: its large values stay below the line.